```
    *
    * @param \Closure(\Illuminate\Http\Request):
    (\Symfony\Component\HttpFoundation\Response) $next
    */
   public function handle(Request $request, Closure $next):
   Response
   {
    if ($request->input('token') !== 'secret-token') {
     return redirect('home');
    }
    return $next($request);
   }
}
```

As you can see, the middleware will send the client an HTTP redirect if the provided token does not match our secret **`token`**; otherwise, the request will be sent to the application. Calling the **`$next`** callback with the **`$reque`**st will transfer the request further into the application, letting the middleware "pass".

The easiest way to think of middleware is as a set of **layers** that HTTP requests must transit through before they can access your application. Each tier has the option to review the request or even reject it completely.

Before or after forwarding the request further into the application, a middleware might do our tasks. For instance, the middleware shown here would carry out a job before the application handles the request:

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;
class BeforeMiddleware
{
 public function handle(Request $request, Closure $next):
 Response
 {
  // Perform action
  return $next($request);
```